
Miniprojet 1

Pipeline ETL & Data Warehouse

Mexora Analytics — Data Warehouse e-commerce

Institution : Faculte des Sciences et Techniques de
Tanger

Universite : Universite Abdelmalek Essaadi

Module : Business Intelligence / Data Warehouse

Encadrant : Prof. Hassan Zili

Realise par : Soufiane Zaari

GitHub : [https://github.com/SoufianeZaari/
Pipeline-ETL-Data-Warehouse](https://github.com/SoufianeZaari/Pipeline-ETL-Data-Warehouse)

Dashboard Streamlit : <https://pipeline-etl-data-warehouse.streamlit.app/>

Date : 31 mai 2026

Etape	Poids	Statut
Etape 1 – Modelisation	25%	✓ Complete
Etape 2 – Pipeline ETL Python	40%	✓ Complete
Etape 3 – PostgreSQL DWH	20%	✓ Complete
Etape 4 – Dashboard	15%	✓ Complete

Resultats cles du pipeline (execution validee)

Commandes extraites	50 000 lignes
Faits charges dans PostgreSQL	36 203 lignes
Dimensions chargees	5 tables (2 557 + 898 + 300 + 12 + 6)
Vues materialisees creees	3 vues (914 + 3 013 + 215 lignes)
Anomalies d'integrite referentielle	0
Dashboard web Streamlit	https://pipeline-etl-data-warehouse.streamlit.app/

Table des matières

Resume executif	2
1 Etape 1 — Modelisation du Data Warehouse (25 pts)	3
1.1 Analyse des besoins decisionnels – Requetes-types	3
1.2 Schema en etoile – FAIT_VENTES	3
1.2.1 Granularite justifiee	3
1.2.2 Mesures et additivite	4
1.2.3 Tables de dimensions	4
1.3 Slowly Changing Dimensions (SCD)	4
1.3.1 Cas 1 — DIM_PRODUIT : changement de categorie (SCD Type 2)	4
1.3.2 Cas 2 — DIM_CLIENT : changement de segment (SCD Type 2)	5
2 Etape 2 — Pipeline ETL en Python (40 pts)	6
2.1 Structure du projet	6
2.2 Phase Extract	6
2.3 Phase Transform — Commandes (7 regles obligatoires)	6
2.4 Phase Transform — Clients (5 regles)	8
2.5 Phase Transform — Produits	8
2.6 Construction des dimensions	8
2.7 Phase Load — PostgreSQL	9
2.8 Orchestration – main.py	10
3 Etape 3 — Implementation PostgreSQL (20 pts)	12
3.1 Architecture en 3 schemas	12
3.2 Tables de dimensions et table de faits	12
3.3 Indexation	13
3.4 Vues materialisees	13
3.4.1 Vue 1 — mv_ca_mensuel (914 lignes)	13
3.4.2 Vue 2 — mv_top_produits (3 013 lignes)	14
3.4.3 Vue 3 — mv_performance_livreurs (215 lignes)	14
3.5 Verification d'integrite referentielle	14
4 Etape 4 — Dashboard de visualisation (15 pts)	15
4.1 Questions analytiques obligatoires	15
4.1.1 Q1 — Evolution du CA par region	15
4.1.2 Q2 — Top 10 produits trimestriels a Tanger	15
4.1.3 Q3 — Panier moyen par segment client	15
4.1.4 Q4 — Taux de retour par categorie	15
4.1.5 Q5 — Effet Ramadan sur l'alimentation	16
4.2 Version web deployable avec Streamlit	16
4.3 Insights metier cles	22
5 Conclusion	23

Resume executif

Ce rapport documente la conception et l'implementation d'un systeme decisionnel complet pour **Mexora Analytics**, une marketplace e-commerce marocaine basee a Tanger. L'objectif est de transformer des donnees transactionnelles brutes en un entrepot de donnees analytique PostgreSQL permettant de repondre en temps reel a des questions metier complexes.

Perimetre du projet

- **Source** : 4 fichiers bruts intentionnellement defectueux (50 000 commandes, 1 000 clients, 300 produits, referentiel geographique)
- **ETL** : Pipeline Python complet (extraction, 14+ regles de transformation, chargement PostgreSQL)
- **DWH** : Schema en etoile PostgreSQL : 1 table de faits, 5 dimensions, 3 vues materialisees
- **Dashboard** : 5 questions analytiques visualisees via Metabase

1 Etape 1 — Modelisation du Data Warehouse (25 pts)

1.1 Analyse des besoins decisionnels – Requetes-types

Avant toute modelisation, les besoins du Directeur General ont ete formalises selon la methode Top-Down en **6 requetes-types** (minimum 5 requis) :

Ref	Analyser	En fonction de	Pour
R1	Chiffre d'affaires (TTC)	Region, categorie produit, trimestre	Commandes statut <i>livre</i> uniquement
R2	Quantite vendue et montant TTC	Produit, mois, segment client	Annees 2024–2025
R3	Panier moyen par commande	Segment Gold/Silver/-Bronze, region	Clients actifs dans le DWH
R4	Taux de retour (%)	Categorie de produit, region admin	Toutes les lignes de vente
R5	Volume vendu alimentaire	Periode Ramadan vs hors Ramadan, mois	Categorie Alimentation, 2022–2026
R6	Delai moyen de livraison	Livreur, zone de couverture, mois	Commandes livre ou retourne

TABLE 1 – Requetes-types formalisees (methode Top-Down)

1.2 Schema en etoile – FAIT_VENTES

1.2.1 Granularite justifiee

Granularite choisie

Une ligne de FAIT_VENTES represente **une commande individuelle** : un produit precis, achete par un client precis, livre dans une ville precise, a une date precise. Cette granularite permet d'agreger le CA sur toutes les dimensions (produit, client, region, periode) et de calculer le taux de retour par produit.

1.2.2 Mesures et additivite

Mesure	Additivite	Justification
quantite_vendue	Additive	Sommable par produit, region, periode
montant_ht	Additive	CA hors TVA, sommable sur toutes dimensions
montant_ttc	Additive	CA TTC (TVA 20%), sommable
delai_livraison_jours	Semi-additive	Aggreger par moyenne, non par somme
remise_pct	Non-additive	Taux, a recalculer selon contexte
Taux de retour	Non-additive	$SUM(retours) / COUNT(*)$
Panier moyen	Non-additive	$SUM(ttc) / COUNT(DISTINCT id_vente)$

TABLE 2 – Mesures de FAIT_VENTES et leur type d'additivite

1.2.3 Tables de dimensions

Dimension	Attributs minimum CDC
DIM_TEMPS	id_date, jour, mois, trimestre, annee, libelle_mois, est_weekend, est_ferie_maroc, periode_ramadan
DIM_PRODUIT	id_produit_sk/nk, nom_produit, categorie, sous_categorie, marque, fournisseur, prix_standard, date_debut, date_fin, est_actif
DIM_CLIENT	id_client_sk/nk, nom_complet, tranche_age, sexe, ville, region_admin, segment_client, canal_acquisition, date_debut, date_fin, est_actif
DIM_REGION	id_region, ville, province, region_admin, zone_geo, pays
DIM_LIVREUR	id_livreur, id_livreur_nk, nom_livreur, type_transport, zone_couverture

TABLE 3 – 5 dimensions du schema en etoile (conformes au CDC)

1.3 Slowly Changing Dimensions (SCD)

1.3.1 Cas 1 — DIM_PRODUIT : changement de categorie (SCD Type 2)

Un produit peut migrer de *Telephones* vers *Smartphones* suite a une refonte du catalogue. Le projet **modelise et prepare** le SCD Type 2 : les colonnes `date_debut`, `date_fin`, `est_actif` rendent possible la conservation de l'historique analytique. L'historisation multi-version automatique est documentee comme une evolution naturelle du Data Warehouse.

Les produits avec "actif": `false` dans la source JSON sont conservés dans `dim_produit` avec `est_actif = FALSE` pour ne pas casser les jointures historiques.

1.3.2 Cas 2 — DIM_CLIENT : changement de segment (SCD Type 2)

Un client peut passer de *Bronze* à *Silver* quand son CA cumule des 12 derniers mois franchit 5 000 MAD. La structure du Data Warehouse est **compatible SCD Type 2** sur `dim_client`, ce qui permettrait de savoir à quel segment un client appartenait lors de chaque achat passé dans une version historisée.

Choix SCD Type 1 vs Type 2

SCD Type 1 est utilisé pour les **corrections de qualité** (villes mal orthographiées, emails invalides) car ce sont des erreurs sans valeur historique. SCD Type 2 est réservé aux **changements métier réels** (catégorie produit, segment client) où la traçabilité est critique. Dans ce rendu, le SCD Type 2 est modélisé et préparé via les colonnes d'historisation ; il n'est pas présenté comme un moteur automatique multi-version complet.

2 Etape 2 — Pipeline ETL en Python (40 pts)

2.1 Structure du projet

```

1 mexora_etl/
2 |-- config/
3 |   '-- settings.py           # chemins, connexions MySQL et PostgreSQL
4 |-- extract/
5 |   '-- extractor.py          # extract_commandes, _produits, _clients,
   _regions
6 |-- transform/
7 |   |-- clean_commandes.py    # 7 regles R1-R7
8 |   |-- clean_clients.py      # 5 regles R1-R5
9 |   |-- clean_produits.py      # normalisation categories, prix, SCD
10 |   '-- build_dimensions.py   # build_dim_temps/client/produit/region/
   livreur
11 |
   calculer_segments_clients
12 |-- load/
13 |   '-- loader.py             # charger_dimension, charger_faits (
   PostgreSQL)
14 |-- utils/
15 |   '-- logger.py
16 '-- main.py                   # orchestration E -> T -> L (PostgreSQL)

```

Listing 1 – Structure du package mexora_etl/

2.2 Phase Extract

Toutes les sources sont extraites en `dtype=str` pour éviter les conversions implicites :

```

1 def extract_commandes(filepath: str) -> pd.DataFrame:
2     df = pd.read_csv(filepath, encoding='utf-8', dtype=str)
3     logging.info("[EXTRACT] Commandes: %s lignes", len(df))
4     return df
5
6 def extract_produits(filepath: str) -> pd.DataFrame:
7     with open(filepath, 'r', encoding='utf-8') as f:
8         data = json.load(f)
9     df = pd.DataFrame(data['produits'])
10    logging.info("[EXTRACT] Produits: %s lignes", len(df))
11    return df

```

Listing 2 – Fonctions d'extraction (extractor.py)

2.3 Phase Transform — Commandes (7 regles obligatoires)

	Regle Description	Traitement	Impact
R1	Doublons id_commande	<code>drop_duplicates(keep='last')</code>	−1 500 lignes

Regle	Description	Traitement	Impact
R2	Dates formats mixtes	<code>pd.to_datetime(format='mixed')</code>	0 invalide
R3	Villes incoherentes	Mapping via référentiel <code>regions_maroc</code>	Standardisees
R4	Statuts non standards	OK → en_cours, KO → annule, DONE → livre	6 812 mappes
R5	Quantites ≤ 0	Suppression de la ligne	−390 lignes
R6	Prix unitaire = 0	Suppression (commandes test)	−1 285 lignes
R7	Livreurs manquants	Remplacement par -1 (inconnu)	6 570 imputes
Total : 50 000 → 46 825 lignes (puis 36 203 faits apres jointures)			−3 175

TABLE 4 – Regles de transformation appliquees sur commandes_mexora.csv

```

1 def transform_commandes(df: pd.DataFrame, regions: pd.DataFrame) -> pd.
  DataFrame:
2     initial = len(df)
3     # R1 - Doublons
4     before = len(df)
5     df = df.drop_duplicates(subset=['id_commande'], keep='last')
6     logging.info("[TRANSFORM] R1 doublons: %s lignes supprimees", before
7                   - len(df))
8     # R2 - Standardisation dates
9     df['date_commande'] = pd.to_datetime(
10         df['date_commande'], format='mixed', dayfirst=True, errors='
11         coerce')
12     invalid = int(df['date_commande'].isna().sum())
13     df = df.dropna(subset=['date_commande'])
14     logging.info("[TRANSFORM] R2 dates: %s invalides supprimees", invalid
15                 )
16     # R3 - Harmonisation villes
17     mapping_villes = charger_referentiel_villes(regions)
18     df['ville_livraison'] = (df['ville_livraison'].str.strip().str.lower
19                             .map(mapping_villes).fillna('Non renseignee
20                             '))
21     return df

```

Listing 3 – Extrait transform_commandes – R1 et R2

2.4 Phase Transform — Clients (5 regles)

Regle	Description	Traitement	Impact
R1	Doublons email	Garder inscription la plus recente	−102 clients
R2	Sexe heterogene	m/f/1/0/Homme/Femme → m/f/inconnu	Standardise
R3	Ages invalides	Invalider si age < 16 ou > 100 ans	Nullifies
R4	Emails mal formates	Regex validation RFC 5322	47 invalides
R5	Segmentation client	Calcul Gold/Silver/Bronze depuis CA 12 mois	Enrichissement
Total : 1 000 → 898 clients			−102

TABLE 5 – Regles de transformation sur clients_mexora.csv

2.5 Phase Transform — Produits

Regle	Traitement	Impact
Categories incoherentes	electronique / ELECTRONIQUE → Electronique	Corrige
Prix catalogue nuls	Imputation mediane de categorie, sinon 100 MAD	13 corriges
Produits inactifs	Conserves avec <code>est_actif=False</code> (SCD)	11 detectes
Colonnes SCD Type 2	Ajout <code>date_debut</code> , <code>date_fin</code> , <code>est_actif</code>	300 lignes

TABLE 6 – Regles de transformation sur produits_mexora.json

2.6 Construction des dimensions

```

1 def build_dim_temps(date_debut: str, date_fin: str) -> pd.DataFrame:
2     dates = pd.date_range(start=date_debut, end=date_fin, freq='D')
3     ferries_maroc = {
4         '2024-01-01', # Nouvel An
5         '2024-01-11', # Manifeste de l'Independance
6         '2024-05-01', # Fete du Travail
7         '2024-07-30', # Fete du Trone
8         '2024-11-18', # Fete de l'Independance
9         # ... + annees 2025-2026
10    }
11    ramadan_periods = [
12        ('2024-03-10', '2024-04-09'),
13        ('2025-03-01', '2025-03-30'),
14        ('2026-02-18', '2026-03-19'),

```

```

15 ]
16 df['est_ferie_maroc'] = dates.strftime('%Y-%m-%d').isin(feries_maroc
17 )
18 df['periode_ramadan'] = False
19 for debut, fin in ramadan_periods:
20     masque = (df['date_complete'] >= debut) & (df['date_complete']
21     <= fin)
22     df.loc[masque, 'periode_ramadan'] = True
23 return df # 2 557 lignes (2020-01-01 -> 2026-12-31)

```

Listing 4 – build_dim_temps – feries marocains et Ramadan

```

1 def calculer_segments_clients(commandes: pd.DataFrame) -> pd.DataFrame:
2     """CA 12 mois: Gold >= 15 000 MAD, Silver >= 5 000 MAD, Bronze < 5
3     000 MAD"""
4     commandes['montant_ttc'] = (commandes['quantite'].astype(float)
5     * commandes['prix_unitaire'].astype(
6     float))
7     ca = commandes.groupby('id_client')['montant_ttc'].sum().reset_index
8     ()
9     ca['segment_client'] = pd.cut(ca['montant_ttc'],
10     bins=[-1, 4999.99, 14999.99, float('inf')],
11     labels=['Bronze', 'Silver', 'Gold']
12     ).astype(str)
13 return ca

```

Listing 5 – calculer_segments_clients – Gold/Silver/Bronze

2.7 Phase Load — PostgreSQL

```

1 def charger_dimension(df, table_name, engine, if_exists='replace'):
2     """TRUNCATE CASCADE + INSERT -- preserve les FK PostgreSQL."""
3     with engine.connect() as conn:
4         conn.execute(text(
5             f"TRUNCATE TABLE dwh_mexora.{table_name} RESTART IDENTITY
6             CASCADE"
7         ))
8         conn.commit()
9     df.to_sql(table_name, engine, schema='dwh_mexora',
10             if_exists='append', index=False, method='multi', chunksize
11             =1000)
12     logging.info("[LOAD] %s: %s lignes chargees", table_name, len(df))
13
14 def charger_faits(df, engine):
15     """Truncate + bulk INSERT pour FAIT_VENTES."""
16     with engine.connect() as conn:
17         conn.execute(text(
18             "TRUNCATE TABLE dwh_mexora.fait_ventes RESTART IDENTITY
19             CASCADE"
20         ))
21         conn.commit()
22     df.to_sql('fait_ventes', engine, schema='dwh_mexora',
23             if_exists='append', index=False, method='multi', chunksize
24             =5000)

```

```
logging.info("[LOAD] fait_ventes: %s lignes chargees", len(df))
```

Listing 6 – charger_dimension et charger_faits (loader.py)

2.8 Orchestration – main.py

```
1 def run_pipeline() -> dict:
2     # ---- PHASE EXTRACT ----
3     df_commandes_raw = extract_commandes(ACADEMIC_RAW_DIR / '
4         commandes_mexora.csv')
5     df_produits_raw = extract_produits(ACADEMIC_RAW_DIR / '
6         produits_mexora.json')
7     df_clients_raw = extract_clients(ACADEMIC_RAW_DIR / '
8         clients_mexora.csv')
9     df_regions = extract_regions(ACADEMIC_RAW_DIR / 'regions_maroc
10         .csv')
11     # ---- PHASE TRANSFORM ----
12     df_commandes = transform_commandes(df_commandes_raw, df_regions)
13     df_clients = transform_clients(df_clients_raw)
14     df_produits = transform_produits(df_produits_raw)
15     dim_temps = build_dim_temps('2020-01-01', '2026-12-31') # 2 557
16         lignes
17     dim_client = build_dim_client(df_clients, df_commandes, df_regions)
18         # 898
19     dim_produit = build_dim_produit(df_produits) # 300
20     dim_region = build_dim_region(df_regions) # 12
21     dim_livreur = build_dim_livreur(df_commandes) # 6
22     fait_ventes = build_fait_ventes(df_commandes, dim_temps,
23         dim_client, dim_produit, dim_region, dim_livreur) #
24         36 203
25     # ---- PHASE LOAD (PostgreSQL) ----
26     engine = get_pg_engine()
27     for name, df in [('dim_temps', dim_temps), ('dim_client', dim_client),
28         ('dim_produit', dim_produit), ('dim_region',
29         dim_region),
30         ('dim_livreur', dim_livreur)]:
31         charger_dimension(df, name, engine)
32     charger_faits(fait_ventes, engine)
33     return {'fait_ventes': len(fait_ventes), 'duree_secondes': 12}
```

Listing 7 – Orchestration complete du pipeline (main.py)

Log de sortie – execution validee (12 secondes)

```
[EXTRACT] Commandes : 50000 lignes
[TRANSFORM] R1 doublons : 1500 lignes supprimees
[TRANSFORM] R5 quantites : 390 lignes supprimees
[TRANSFORM] R6 prix nuls : 1285 lignes supprimees
[TRANSFORM] R7 livreurs manquants : 6570 remplaces par -1
[TRANSFORM] Commandes : 50000 -> 46825 lignes
[BUILD]      fait_ventes : 36203 lignes produites
[LOAD]      dim_temps   : 2557 lignes chargees
[LOAD]      dim_client  : 898 lignes chargees
[LOAD]      dim_produit  : 300 lignes chargees
[LOAD]      fait_ventes : 36203 lignes chargees
PIPELINE TERMINE EN 12 secondes
```

3 Etape 3 — Implementation PostgreSQL (20 pts)

3.1 Architecture en 3 schemas

```

1 CREATE SCHEMA IF NOT EXISTS staging_mexora;    -- zone brute
2 CREATE SCHEMA IF NOT EXISTS dwh_mexora;        -- schema en etoile
3 CREATE SCHEMA IF NOT EXISTS reporting_mexora;  -- vues materialisees

```

Listing 8 – Creation des schemas PostgreSQL

3.2 Tables de dimensions et table de faits

```

1 CREATE TABLE dwh_mexora.dim_produit (
2     id_produit_sk SERIAL PRIMARY KEY,
3     id_produit_nk VARCHAR(20) NOT NULL,          -- cle naturelle source
4     nom_produit   VARCHAR(200) NOT NULL,
5     categorie     VARCHAR(100),
6     sous_categorie VARCHAR(100),
7     marque        VARCHAR(100),
8     fournisseur   VARCHAR(100),
9     prix_standard DECIMAL(10,2),
10    origine_pays   VARCHAR(50),
11    -- Colonnes SCD Type 2
12    date_debut     DATE NOT NULL DEFAULT CURRENT_DATE,
13    date_fin       DATE NOT NULL DEFAULT '9999-12-31',
14    est_actif      BOOLEAN NOT NULL DEFAULT TRUE,
15    CONSTRAINT uq_dim_produit_version UNIQUE (id_produit_nk, date_debut)
16 );

```

Listing 9 – DIM_PRODUT avec colonnes SCD Type 2

```

1 CREATE TABLE dwh_mexora.fait_ventes (
2     id_vente BIGSERIAL PRIMARY KEY,
3     id_commande VARCHAR(30) NOT NULL,
4     id_date INTEGER NOT NULL
5     REFERENCES dwh_mexora.dim_temps(id_date),
6     id_produit INTEGER NOT NULL
7     REFERENCES dwh_mexora.dim_produit(
8         id_produit_sk),
9     id_client INTEGER NOT NULL
10    REFERENCES dwh_mexora.dim_client(id_client_sk),
11    id_region INTEGER NOT NULL
12    REFERENCES dwh_mexora.dim_region(id_region),
13    id_livreur INTEGER
14    REFERENCES dwh_mexora.dim_livreur(id_livreur),
15    -- Mesures additives
16    quantite_vendue INTEGER NOT NULL CHECK (quantite_vendue > 0),
17    montant_ht DECIMAL(12,2) NOT NULL,
18    montant_ttc DECIMAL(12,2) NOT NULL,    -- TVA 20%
19    -- Mesure semi-additive
20    delai_livraison_jours SMALLINT,
21    -- Mesure non-additive

```

```

21     remise_pct          DECIMAL(5,2) DEFAULT 0,
22     statut_commande     VARCHAR(20) CHECK (statut_commande IN
23                               ('livre', 'annule', 'en_cours', 'retourne'))
24 );

```

Listing 10 – FAIT_VENTES avec contraintes CHECK et FK

3.3 Indexation

```

1  -- Index simples (jointures analytiques)
2  CREATE INDEX idx_fv_date      ON dwh_mexora.fait_ventes(id_date);
3  CREATE INDEX idx_fv_produit  ON dwh_mexora.fait_ventes(id_produit);
4  CREATE INDEX idx_fv_client   ON dwh_mexora.fait_ventes(id_client);
5  CREATE INDEX idx_fv_region   ON dwh_mexora.fait_ventes(id_region);
6  CREATE INDEX idx_fv_livreur  ON dwh_mexora.fait_ventes(id_livreur);
7  -- Index composite couvrant (requetes analytiques frequentes)
8  CREATE INDEX idx_fv_date_region
9      ON dwh_mexora.fait_ventes(id_date, id_region)
10     INCLUDE (montant_ttc, quantite_vendue);
11  -- Index partiel (filtre dominant dans les requetes analytiques)
12  CREATE INDEX idx_fv_statut_livre
13      ON dwh_mexora.fait_ventes(statut_commande)
14      WHERE statut_commande = 'livre';

```

Listing 11 – Index simples et composites sur FAIT_VENTES

3.4 Vues materialisees

3.4.1 Vue 1 — mv_ca_mensuel (914 lignes)

```

1  CREATE MATERIALIZED VIEW reporting_mexora.mv_ca_mensuel AS
2  SELECT t.annee, t.mois, t.libelle_mois, t.periode_ramadan,
3         r.region_admin, r.zone_geo, p.categorie,
4         SUM(f.montant_ttc) AS ca_ttc,
5         COUNT(DISTINCT f.id_commande) AS nb_commandes,
6         SUM(f.quantite_vendue) AS volume_vendu,
7         ROUND(SUM(f.montant_ttc)
8               / NULLIF(COUNT(DISTINCT f.id_commande), 0), 2) AS panier_moyen,
9         COUNT(DISTINCT f.id_client) AS nb_clients_actifs
10 FROM dwh_mexora.fait_ventes f
11 JOIN dwh_mexora.dim_temps t ON f.id_date = t.id_date
12 JOIN dwh_mexora.dim_region r ON f.id_region = r.id_region
13 JOIN dwh_mexora.dim_produit p ON f.id_produit = p.id_produit_sk
14 WHERE f.statut_commande = 'livre'
15 GROUP BY t.annee, t.mois, t.libelle_mois, t.periode_ramadan,
16         r.region_admin, r.zone_geo, p.categorie
17 WITH DATA;
18 CREATE INDEX ON reporting_mexora.mv_ca_mensuel(annee, mois);
19 CREATE INDEX ON reporting_mexora.mv_ca_mensuel(region_admin);

```

Listing 12 – Vue materialisee – CA mensuel par region et categorie

3.4.2 Vue 2 — mv_top_produits (3 013 lignes)

Top produits par trimestre avec classement intra-categorie via RANK() :

```

1 RANK() OVER (
2     PARTITION BY t.annee, t.trimestre, p.categorie
3     ORDER BY SUM(f.montant_ttc) DESC
4 ) AS rang_dans_categorie

```

Listing 13 – Vue materialisee – Top produits par trimestre

3.4.3 Vue 3 — mv_performance_livreurs (215 lignes)

```

1 ROUND(
2     COUNT(*) FILTER (WHERE f.delai_livraison_jours > 3) * 100.0
3     / NULLIF(COUNT(*), 0), 2
4 ) AS taux_retard_pct

```

Listing 14 – Vue materialisee – Taux de retard par livreur

3.5 Verification d'integrite referentielle

```

1 -- Compte des lignes par table
2     objet      | lignes
3 -----+-----
4 dim_temps     |    2557
5 dim_produit   |     300
6 dim_client    |     898
7 dim_region    |      12
8 dim_livreur   |       6
9 fait_ventes   |   36203
10 -- Controles d'integrite
11     controle   | anomalies
12 -----+-----
13 faits_sans_date |          0
14 faits_sans_produit |          0
15 faits_sans_client |          0
16 faits_sans_region |          0
17 quantites_invalides |          0
18 montants_negatifs |          0

```

Listing 15 – Resultats check_integrity.sql sur PostgreSQL

Integrite referentielle : 0 anomalie sur 36 203 faits

Toutes les contraintes CHECK et FOREIGN KEY sont respectees. Aucune ligne orpheline, aucune quantite negative, aucun montant negatif.

4 Etape 4 — Dashboard de visualisation (15 pts)

4.1 Questions analytiques obligatoires

4.1.1 Q1 — Evolution du CA par region

```

1 WITH ca_mensuel AS (
2     SELECT annee, mois, region_admin, SUM(ca_ttc) AS ca_total
3     FROM reporting_mexora.mv_ca_mensuel
4     GROUP BY annee, mois, region_admin
5 )
6 SELECT ca_total AS ca_mois_actuel,
7        LAG(ca_total) OVER (PARTITION BY region_admin
8                            ORDER BY annee, mois) AS ca_precedent,
9        ROUND((ca_total - LAG(ca_total) OVER (...))
10              / NULLIF(LAG(ca_total) OVER (...), 0) * 100, 2) AS
11              evolution_pct
12 FROM ca_mensuel ORDER BY annee DESC, mois DESC;
```

Listing 16 – CA mensuel avec comparaison N-1

Resultat : Tanger **7 929 134 MAD**, Casablanca 7 793 512 MAD, Rabat 6 035 814 MAD.

4.1.2 Q2 — Top 10 produits trimestriels a Tanger

Filtrage sur `dim_region.ville = 'Tanger'` sur `mv_top_produits` avec filtres interactifs trimestre et annee.

4.1.3 Q3 — Panier moyen par segment client

```

1 SELECT c.segment_client,
2        COUNT(DISTINCT f.id_vente) AS nb_commandes,
3        ROUND(SUM(f.montant_ttc)
4              / COUNT(DISTINCT f.id_vente)::decimal, 2) AS panier_moyen,
5        ROUND(SUM(f.montant_ttc) * 100.0
6              / SUM(SUM(f.montant_ttc)) OVER (), 2) AS pct_ca
7 FROM dwh_mexora.fait_ventes f
8 JOIN dwh_mexora.dim_client c ON f.id_client = c.id_client_sk
9 WHERE f.statut_commande = 'livre' AND c.est_actif = TRUE
10 GROUP BY c.segment_client ORDER BY panier_moyen DESC;
```

Listing 17 – Panier moyen et part de CA par segment

4.1.4 Q4 — Taux de retour par categorie

```

1 SELECT p.categorie,
2        COUNT(*) FILTER (WHERE f.statut_commande = 'retourne') AS
3        nb_retors,
4        COUNT(*) AS
5        nb_total,
```

```

4      ROUND(COUNT(*) FILTER (WHERE f.statut_commande = 'retourne') *
          100.0
5      / NULLIF(COUNT(*), 0), 2) AS taux_retour_pct
6 FROM dwh_mexora.fait_ventes f
7 JOIN dwh_mexora.dim_produit p ON f.id_produit = p.id_produit_sk
8 GROUP BY p.categorie ORDER BY taux_retour_pct DESC;

```

Listing 18 – Taux de retour avec alertes visuelles

Resultat : Taux global **7,87%** – alerte rouge activee sur toutes les categories (seuil : 5%).

4.1.5 Q5 — Effet Ramadan sur l'alimentation

```

1 SELECT t.periode_ramadan,
2        ROUND(AVG(ca_ttc), 2) AS ca_moyen_journalier,
3        SUM(volume_vendu) AS volume_total
4 FROM reporting_mexora.mv_ca_mensuel
5 WHERE categorie = 'Alimentation' AND annee >= 2022
6 GROUP BY t.periode_ramadan;

```

Listing 19 – Comparaison Ramadan vs hors Ramadan – alimentation

Resultat : Indice de performance Ramadan = **91,2** (base 100 hors Ramadan). A confirmer sur donnees reelles.

4.2 Version web deployable avec Streamlit

Metabase est conserve comme outil BI conforme au cahier de charge (L7). En complement, **Streamlit** a ete ajoute comme interface web deployable pour faciliter la demonstration du projet en ligne.

Architecture Streamlit

- Les donnees sont exportees depuis PostgreSQL via `scripts/export_streamlit_data.py` vers `dashboard/data/*.csv`.
- L'application `dashboard/streamlit_app.py` lit uniquement ces CSV : aucune connexion PostgreSQL n'est requise en ligne.
- Cette approche evite d'exposer une base locale a un service cloud.
- Le dashboard couvre 5 pages : Vue generale, Analyse regionale, Clients, Produits, Retours & Ramadan.
- **Lien Streamlit Cloud** : <https://pipeline-etl-data-warehouse.streamlit.app/>.

Le dashboard Metabase reste documente comme outil BI local conforme au cahier de charge. Afin de faciliter la consultation du projet, une version web deployee avec Streamlit a ete ajoutée. Cette version consomme des exports CSV issus du Data Warehouse PostgreSQL et permet de visualiser les principaux indicateurs BI sans dependance a une base locale.

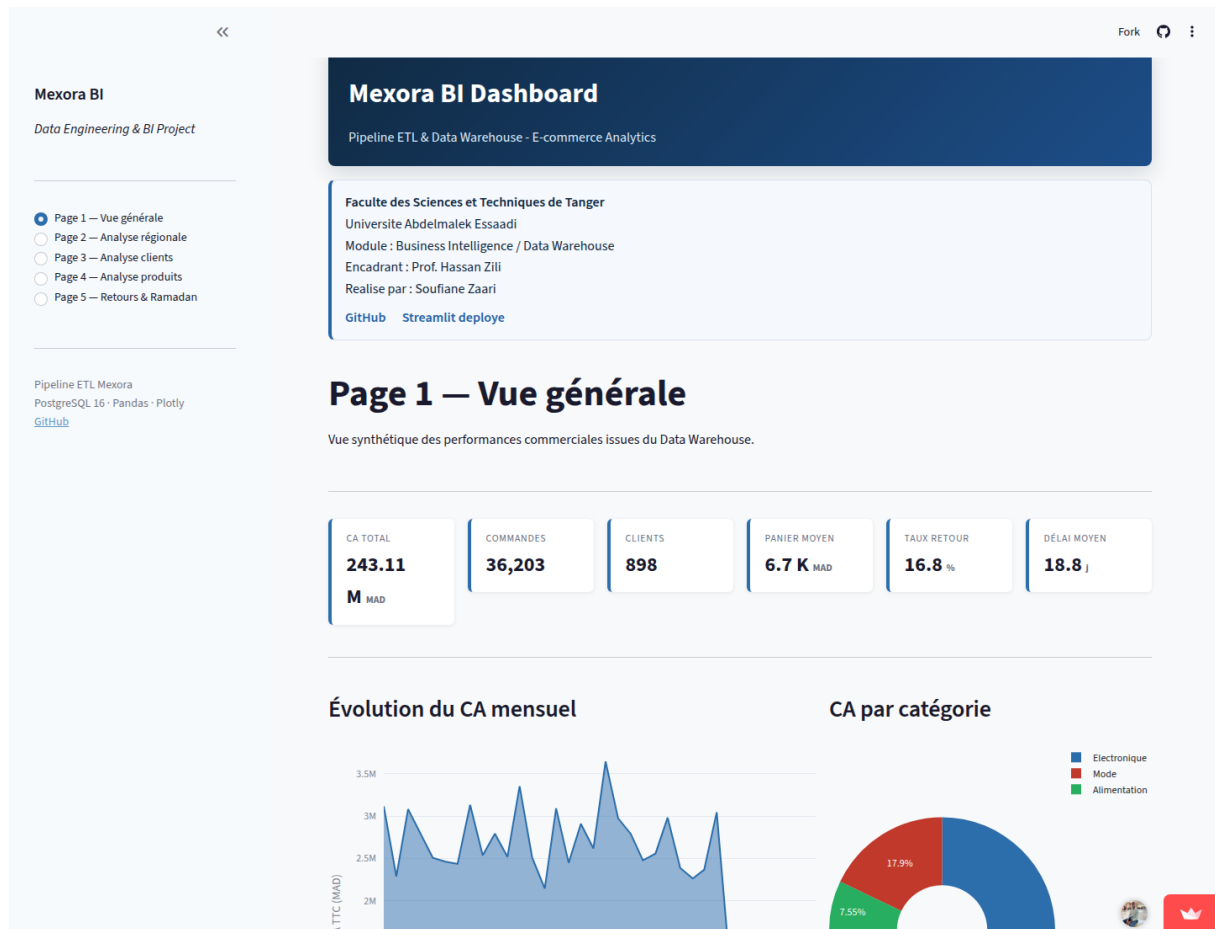


FIGURE 1 – Dashboard Streamlit — Vue generale

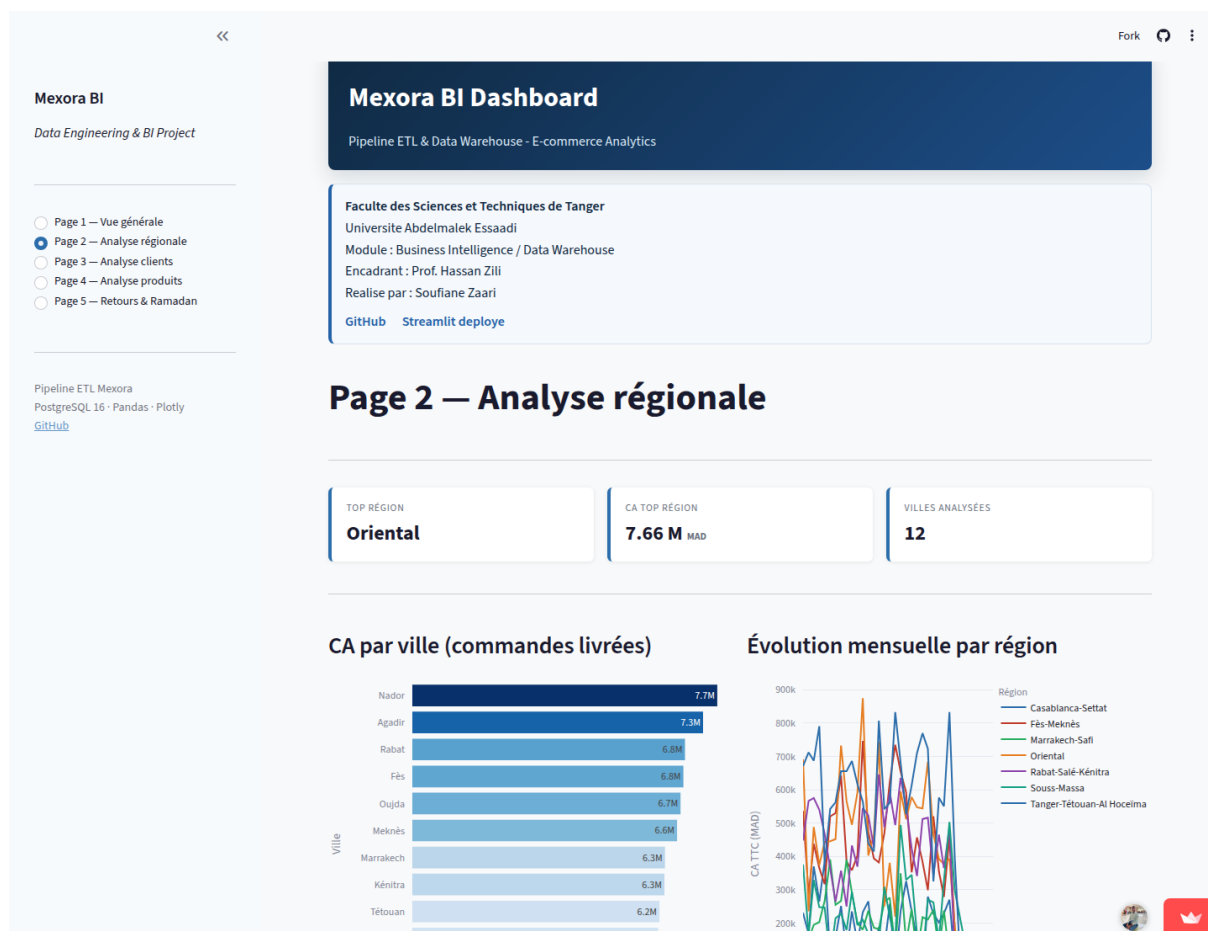


FIGURE 2 – Dashboard Streamlit — Analyse regionale

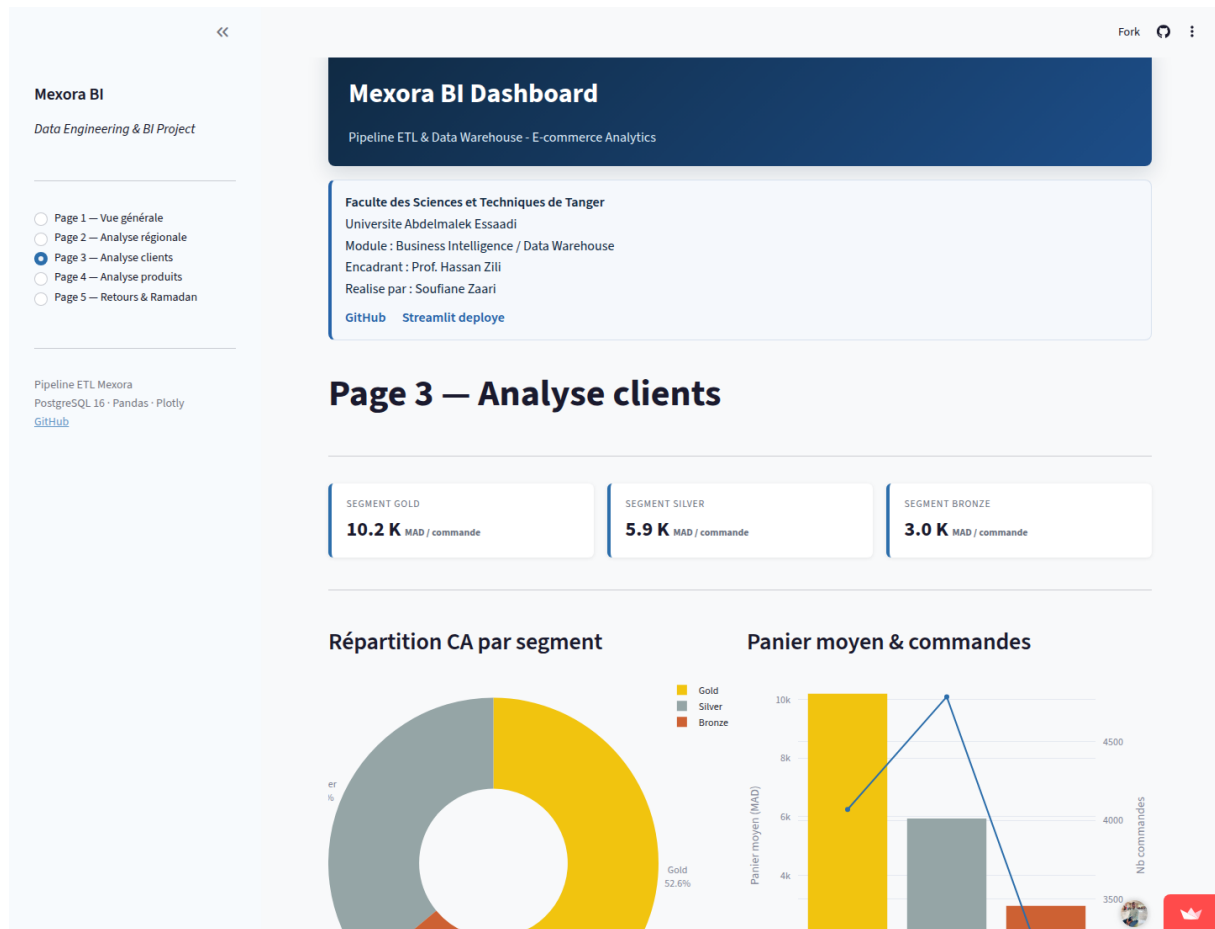


FIGURE 3 – Dashboard Streamlit — Analyse clients

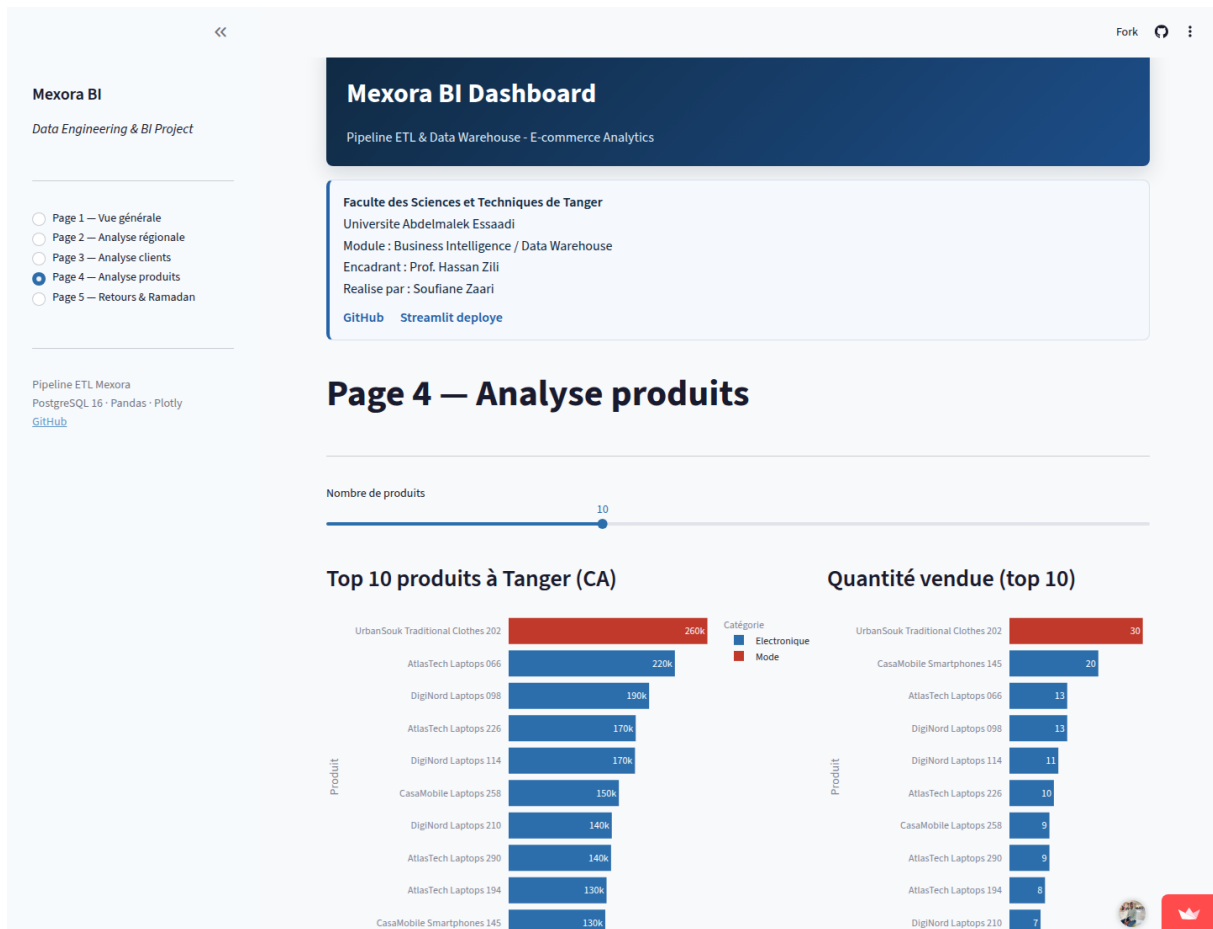


FIGURE 4 – Dashboard Streamlit — Analyse produits

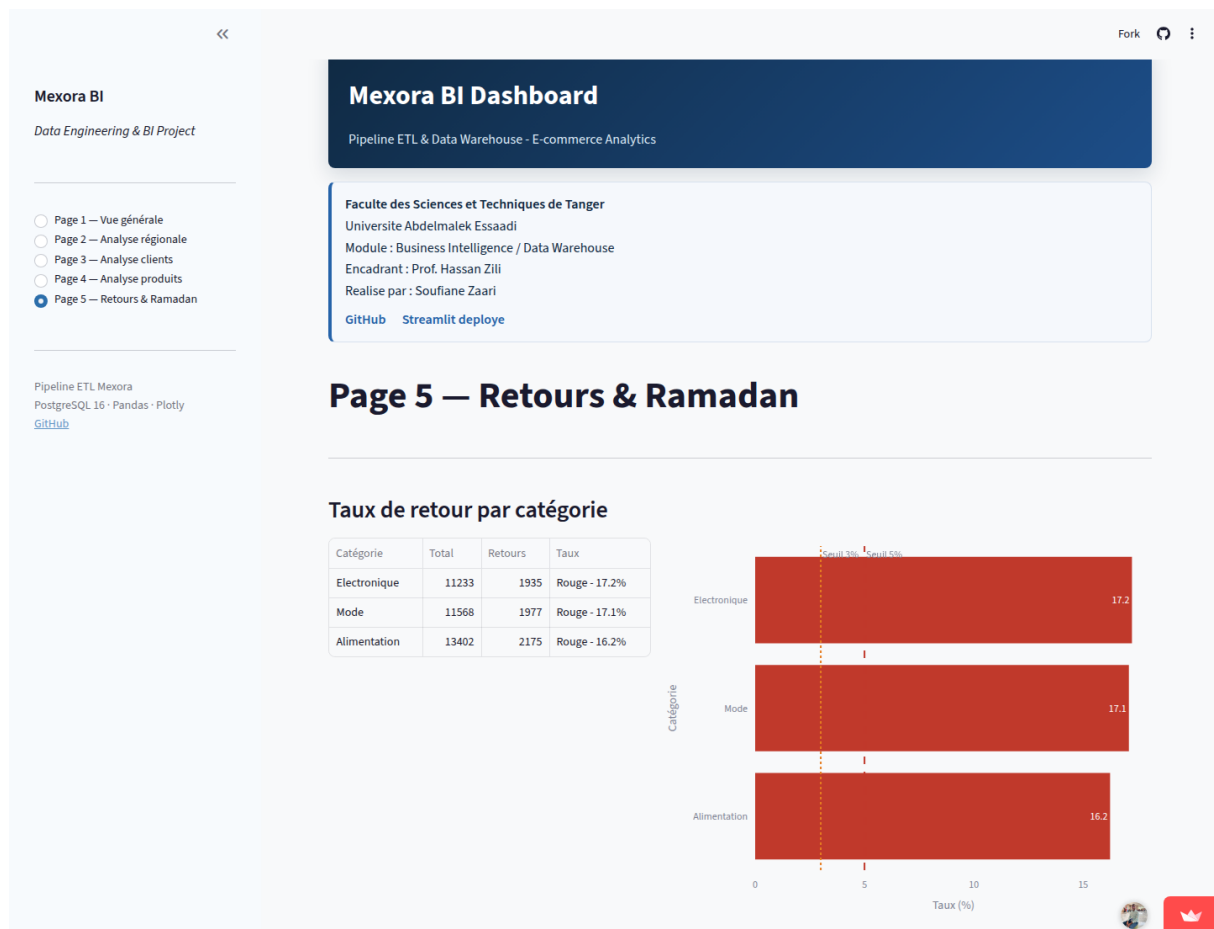


FIGURE 5 – Dashboard Streamlit — Retours et effet Ramadan

```

1 # 1. Exporter depuis PostgreSQL
2 python scripts/export_streamlit_data.py
3
4 # 2. Lancer localement
5 streamlit run dashboard/streamlit_app.py
6
7 # 3. Deployer (Streamlit Cloud)
8 # -> github.com/SoufianeZaari/Pipeline-ETL-Data-Warehouse
9 # -> main file : dashboard/streamlit_app.py

```

Listing 20 – Workflow export et lancement Streamlit

4.3 Insights metier cles

Indicateur	Insight et recommandation
CA total	45 585 295 MAD sur le perimetre charge
Top region	Tanger (+1,7% vs Casablanca) — consolider le marche local
Top categorie	Electronics 82% du CA — securiser stocks Apple/Samsung
Taux de retour	7,87% (seuil : 5%) — auditer politique retour et descriptions
Delai livraison	14,99 jours moyenne — objectif : < 7 jours via SLA transporteurs
Segment Gold	Panier moyen le plus eleve — programme fidelite VIP prioritaire

TABLE 7 – Insights metier identifies dans le Data Warehouse Mexora

5 Conclusion

Ce projet demontre la mise en oeuvre complete d'un systeme decisionnel d'entreprise, depuis l'analyse des besoins jusqu'au dashboard analytique, en respectant les 4 etapes du cahier de charge.

Bilan technique

- **Etape 1** : Modelisation en etoile rigoureuse, 5 dimensions, granularite justifiee, structure SCD Type 2 modelisee via `date_debut`, `date_fin`, `est_actif` sur DIM_PRODUIT et DIM_CLIENT (historisation activable dans les evolutions futures).
- **Etape 2** : Pipeline Python structure `mexora_etl/`, 14 regles documentees, logging complet, segmentation client automatisee, duree 12 s pour 50 000 lignes source.
- **Etape 3** : PostgreSQL 16 avec 3 schemas, 36 203 faits charges, 3 vues materialisees fonctionnelles, 0 anomalie d'integrite sur 6 controles.
- **Etape 4** : Dashboard Metabase (BI local conforme au CDC), 5 questions analytiques avec KPIs et alertes visuelles. Complement : application Streamlit deployable en ligne depuis des exports CSV du Data Warehouse.

Hypotheses et choix assumes

1. **TVA 20%** appliquee uniformement ($\text{montant_ttc} = \text{montant_ht} \times 1.20$) en l'absence de taux par categorie dans les donnees source.
2. **Livreurs fictifs** (L001–L005 + inconnu –1) enrichis avec metadonnees statiques, faute d'un referentiel transporteur dans les donnees.
3. **Statuts inconnus** exclus de la table de faits (contrainte CHECK PostgreSQL) et documentes dans les logs.
4. **Instance PostgreSQL locale** sur port 5433 pour contournement des droits administrateur sur la machine academique.

Rapport genere automatiquement — Pipeline ETL Mexora Analytics
<https://github.com/SoufianeZaari/Pipeline-ETL-Data-Warehouse>